EECS 221: Machine Learning for Science

Data–Compute Trade-offs in Neural PDE Forecasting

Suha Kagan Kose

Mid-Project Report

Project Goal. Neural PDE forecasting models must be useful under limited data and compute. This project compares how accurately and efficiently three architecture families (CNextU-Net, TFNO and AViT) learn Rayleigh–Bénard convection when training data and GPU time are limited. This check-in covers the most constrained setting of the study: 10% of the training data (140 trajectories, four per regime) and 1 or 4 hours of training on one A100 GPU. The 100% runs is planned to be conducted in the following stage of the project as indicated in (Section 6).

Milestones. The abstract set three milestones for this check-in; all of these were carried out as planned.

1. **Data pipeline validated against The Well's released checkpoints.** Our data loading and metric code reproduce the paper's published test VRMSE for the released CNextU-Net and TFNO to within 1%, and recover rollout errors in the same reported range ("> 10"). The released TFNO needs the forward pass it was trained with, which differs from the corrected one our runs use (Section 1, Appendix C).
2. **Budgets fixed from pilot timings.** Batch 64, fp32, peak learning rate 10^{-3} for every model (from an equal-budget search), and 4-hour budgets measured in active GPU training time, with allocated GPU time reported separately.
3. **First 10%-data results.** All six 10% runs (3 models $\times$ 2 seeds) are trained and evaluated on the full test set at 1 and 4 hours, together with what each model costs to train and to use (Section 4). A revised plan is in Section 6.

1 Setup

Data and task. `rayleigh_benard` from The Well [1]: 1,750 trajectories of 200 frames on a 512×128 grid, 35 (Rayleigh, Prandtl) regimes, official 80/10/10 splits. From four frames of buoyancy, pressure and both velocity components, a model predicts the next frame and is rolled out autoregressively; consecutive frames are 0.25 simulation time units apart [13]. Fields are normalised with The Well's training statistics. The study trains on nested subsets of the training split; this report uses the 10% subset (140 of the 1,400 training trajectories, four of the 40 in each regime) and evaluates on the full test split.

Models. The Well's own implementations, trained from random initialisation, with matched parameter counts (Table 1). All three use GELU activations. For TFNO, released-checkpoint reproduction and training with the current code must be distinguished: the paper-era code skipped TFNO's Fourier layers, an error fixed upstream in November 2025. We scored the released TFNO weights with the forward pass they were trained with, and our own TFNO, trained with the fixed code (The Well at `e63dec8`), uses its Fourier layers (Appendix C).

Training. All models use AdamW, one-step MSE in normalised field values, batch size 64 and fp32, with a peak learning rate of 10^{-3}, 500 warmup steps and cosine decay to zero over four hours of active

Model	Family	Parameters	Normalisation	Relation to released checkpoint
CNextU-Net [2]	ConvNeXt U-Net, 4 stages	18.58M	LayerNorm	same architecture
TFNO [5, 3]	Fourier operator, 16×16 modes	19.28M	none	same configuration, fixed forward
AViT [4]	axial-attention ViT	18.57M	GroupNorm, LayerNorm	none released; width 384 → 432

Table 1: The three model families.

GPU training on one A100. The one- and four-hour budgets are the study's compute axis. The paper trained for 12 hours on an H100 with its own learning rates and schedule, so ours is a different experiment (Appendix A). Checkpoints are saved at one and four hours. The one-hour results are those checkpoints after one hour of the four-hour schedule, with the learning rate still at about 85% of its peak: they show what each model has learned after one hour of training, not the best a schedule ending at one hour could reach. Each configuration is trained with two seeds, which change the initialisation and the order of the training data. All runs share one 10% subset, so the spread between seeds does not include the effect of which four trajectories per regime were drawn, which may be larger. With two seeds, differences within the seed range are not read as rankings.

VRMSE. For one window (one predicted frame) and one field f, The Well's metric code (its `VRMSE` class at commit `e63dec8`) computes

$$\mathrm{VRMSE}_f = \sqrt{\frac{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_{f,i} - y_{f,i})^2}{s_f^2 + \varepsilon}}, \qquad s_f^2 = \frac{1}{N-1}\sum_{i=1}^{N}(y_{f,i} - \bar{y}_f)^2, \qquad \varepsilon = 10^{-7},$$

over the $N = 512 \times 128$ grid points of the target frame. The four fields (buoyancy b, pressure p, and the velocity components u, w) are averaged with equal weights, and reported values are means over windows with equal weight per window (one-step: all 196 windows of each of the 175 test trajectories; rollouts: per step, over trajectories and start frames). Two variants use the same formula and averaging and differ only in the field values passed in: the *paper protocol* scores normalised fields, $(y_f - \mu_f)/\sigma_f$ with The Well's training statistics ($\sigma_f = 0.249, 0.149, 0.208, 0.180$ for b, p, u, w); our *physical-field VRMSE* scores the fields as stored. Appendix B explains how and where the two differ.

Failure horizon. For a rollout, the first step whose field-mean physical-field VRMSE exceeds a threshold (1, 2 or 5, all reported). Rollouts start at frames 20, 80 and 140 of every test trajectory and run 30 steps. We report the fraction of rollouts that stay below each threshold through step 30 (Table 4) and, for threshold 1, its value at every step (Kaplan–Meier survival; Fig. 3b).

The kinetic-energy spectrum and the inference-time measurements are defined in Appendix B.

2 Milestone 1: Pipeline Validated Against the Released Checkpoints

We loaded The Well's released CNextU-Net and TFNO weights and scored them with our own data path and metric on every window of the 175 test trajectories (Table 2). The released TFNO also reproduces its published value, but only with the forward pass it was trained with. A bug in The Well's code at the time skipped TFNO's Fourier layers, so the released model is effectively a per-pixel network: each output pixel depends only on the same pixel's input frames, and it cannot move structures across the grid. It therefore learned little more than copying the last frame, and its error tracks persistence (Fig. 1): slightly lower in developed convection, much higher during onset. Its full results are in Appendix C.

Three observations shape how we report results (the second and third are in Appendix D):

	Paper-protocol one-step VRMSE		Rollout from frame 0, steps 6–12 / 13–30	
Model	Ours (released weights)	Published	Ours	Published
CNextU-Net	0.669	0.670	36 / 35	> 10 / > 10

Table 2: The released CNextU-Net scored by our pipeline against the paper's Tables 2 and 3 (test split). The released TFNO is in Appendix C (Table 9).

- **Aggregate relative error is sensitive to the early frames** (Fig. 1). For the first ∼50 frames the fluid is nearly at rest; the evaluated models have large relative velocity errors there, and copying the last frame (persistence) is nearly exact. From frame 50 on, the released CNextU-Net scores 0.038, against an all-window mean of 0.224 in physical fields (0.669 under the paper protocol). Near-zero target variability can amplify relative error, so this does not establish that absolute errors are largest early on. We therefore also report early (*onset*) and later (*developed*) windows separately.

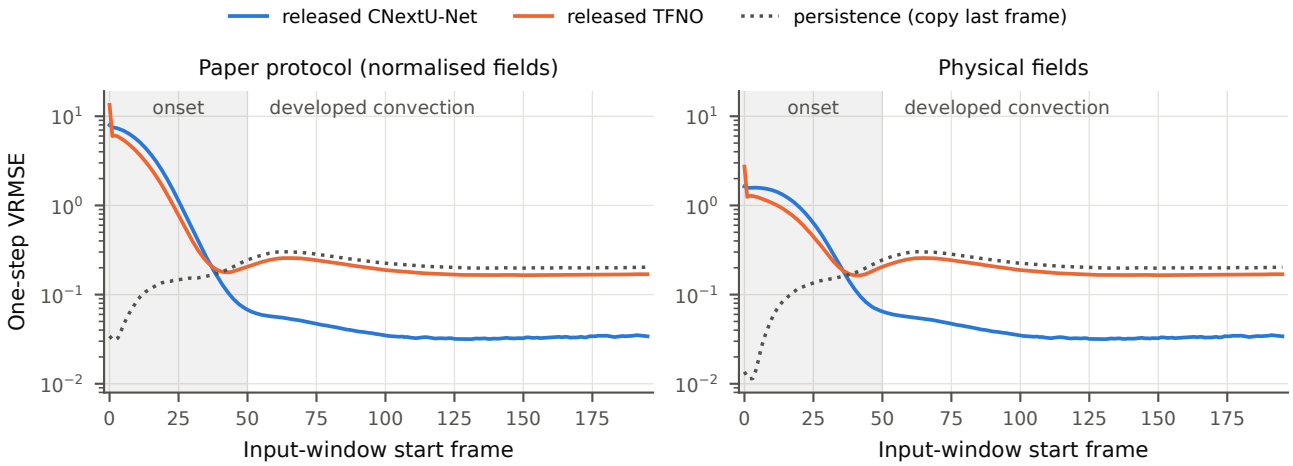


Figure 1: Released checkpoints' one-step VRMSE by window on the test set, mean over the 175 trajectories, under the paper protocol (left) and on physical fields (right). Shaded: windows whose first input frame is below 50, reported as *onset*; the others are *developed*. The cut at frame 50 is operational, chosen where the released checkpoints' error levels off on the validation split; it is not a transition verified for every regime. Persistence copies the last input frame. The released TFNO tracks persistence because of the paper-era bug (Section 2, Appendix C).

3 Milestone 2: Budgets Determined from Pilot Timings

Resource accounting. *Active GPU training time* is the measured training time counted against the budget (optimizer steps only; Appendix E gives the timer boundaries). *Allocated GPU time* is the duration for which a job occupies a GPU, including data waits and other overhead. *Service units (SU)* are the cluster's charges on its paid partition. Queue time is reported separately and is not allocated GPU time.

Budget clock. Data are read from the cluster's shared file system, which delivers at most ∼98 training samples per second into one node and stalls for 5–22 minutes at a time, on all nodes at once. TFNO and AViT consume 116 and 330 samples per second of GPU time (Fig. 7), so a wall-clock budget would partly measure our storage. The 1-hour and 4-hour budgets therefore count active GPU training time; waiting for data is logged separately.

Learning rate. The paper's rates ($5 \cdot 10^{-4}$ for CNextU-Net, 10^{-4} for TFNO) were chosen for 12-hour

runs. With one to four hours a model takes far fewer optimizer steps, and a small learning rate that pays off over a long run may not get far enough in a short one, so we tested whether a higher rate suits our budgets. An equal-budget search (20 minutes of GPU training per run, 10% data, validation only) chose 10^{-3} for all three models (Table 8, Appendix A). At this budget the higher rate was clearly better than the paper's: 0.408 against 0.469 for CNextU-Net at $5 \cdot 10^{-4}$, and 0.302 against 0.603 for TFNO at 10^{-4}. The Well swept $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$; we tested its lower three rates and omitted the two largest to limit tuning cost. Since 10^{-3} is the largest rate tried, it is the best candidate tested, not an established optimum, and the 20-minute ranking may differ from the ranking after four hours.

Batch size and precision. Batch 64 (The Well's) fits every model in fp32 on an A100 80 GB; CNextU-Net peaks at 64.8 GB of memory reserved by PyTorch (60.9 GB allocated, Table 5). The current TFNO implementation requires fp32 FFT operations, so all models use fp32 for this comparison. This is an implementation constraint, not a claim that mixed-precision TFNO is impossible. The choice favours TFNO: in the batch-64 profile, bf16 cuts the GPU time of a training step by a factor of 1.7 for CNextU-Net and 2.2 for AViT, and CNextU-Net's memory from 58.7 to 34.5 GB. With mixed precision both would take more optimizer steps in the same budget; AViT would also need faster storage to benefit, since it already waits for data.

4 Milestone 3: First Results Under Limited Data and Compute

This section answers the project's question for its most constrained setting: how accurate each family becomes with 10% of the training data and 1 or 4 hours of training, and what it costs to train and to use.

Training (Fig. 2). The six runs completed their 4-hour budgets (Table 10). On a fixed validation sample of 64 windows (Appendix E), AViT and CNextU-Net reach similar error after 4 hours; TFNO's loss plateaus after ∼1.5 hours. CNextU-Net's validation error peaks near one hour while its learning rate is still high; this temporal association does not establish the learning rate as the cause.

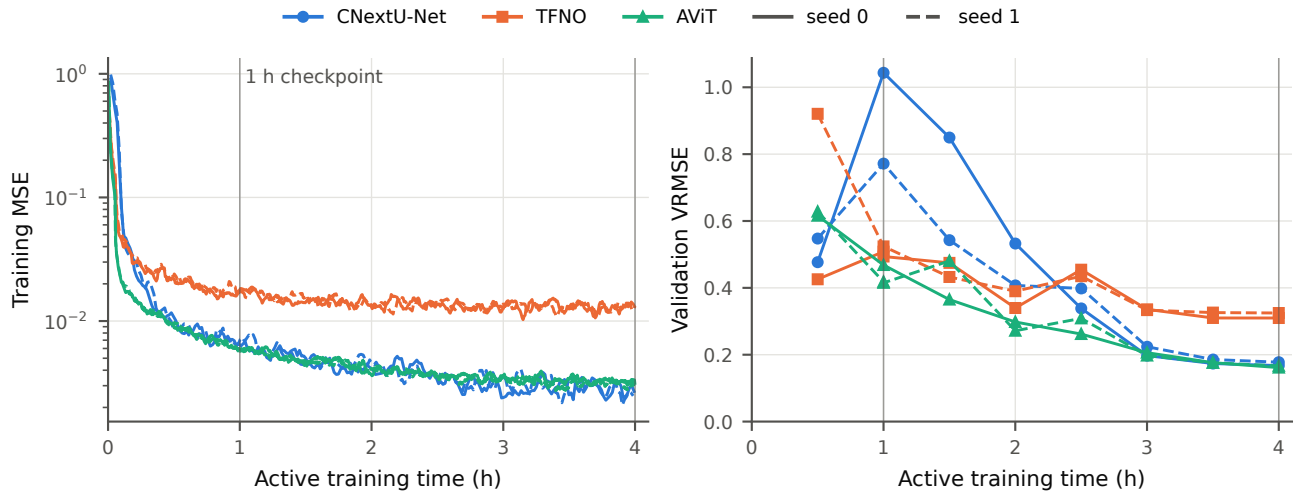


Figure 2: Training MSE in normalised units (left; smoothed over 3 minutes of training) and validation VRMSE on the 64 fixed windows, physical fields (right), against active training time, 10% data. Solid: seed 0; dashed: seed 1. Vertical lines: the 1-hour and 4-hour checkpoints.

Test evaluation (all 175 test trajectories, both seeds; Table 3, Fig. 8 in Appendix F). After 4 hours AViT and CNextU-Net have similar one-step error over all windows (0.134 and 0.136) and TFNO a higher one (0.209), but the two phases separate them: in developed convection CNextU-Net is clearly the

most accurate (0.050, against 0.075 for AViT and 0.109 for TFNO), consistent with U-Net variants being among the strongest baselines in neural PDE benchmarks [6, 1], while AViT is the most accurate during onset. The across-seed standard deviation of the all-window VRMSE is at most 0.006 (seeds only; the subset is fixed). The released CNextU-Net, trained on all the data for longer, remains better in developed convection (0.038 against 0.050). This gap is not a measure of what ten times more data buys, because the two also differ in hardware, learning rate and schedule (Appendix A). The 100% runs will measure the effect of data with everything else fixed. Persistence is better than every model during onset and 2–4.5 times worse in developed convection.

	Physical-field one-step VRMSE			Paper protocol
Model (10% data, checkpoint)	all windows	onset	developed	(all windows)
CNextU-Net, 4 h	0.136 ± 0.006	0.387 ± 0.025	**0.050** ± 0.000	0.349 ± 0.022
TFNO, 4 h	0.209 ± 0.001	0.502 ± 0.003	0.109 ± 0.000	0.488 ± 0.001
AViT, 4 h	**0.134** ± 0.001	**0.308** ± 0.003	0.075 ± 0.000	**0.298** ± 0.002
CNextU-Net, 1 h	0.728 ± 0.176	2.618 ± 0.681	**0.081** ± 0.003	2.345 ± 0.600
TFNO, 1 h	0.473 ± 0.078	1.505 ± 0.305	0.120 ± 0.000	1.362 ± 0.249
AViT, 1 h	**0.394** ± 0.036	**1.299** ± 0.140	0.085 ± 0.000	**1.171** ± 0.125
Released CNextU-Net (100%, paper)	0.224	0.766	0.038	0.669
Persistence	0.199	0.121	0.226	0.202

Table 3: One-step test error: mean ± standard deviation over the two seeds (0.000: below 0.0005). Onset: first input frame < 50; developed: ≥ 50. Bold: best of our models at each budget.

Rollouts (Table 4, Fig. 3). One-step accuracy does not carry over to rollouts in the same order. This is expected for models trained only on one-step targets: in a rollout they receive their own slightly wrong outputs, inputs they never saw in training [9], and uncontrolled error growth over an autoregressive rollout is a known failure mode [10]. Among our trained models, CNextU-Net is the most stable: after 30 steps from frames 80 and 140 its error (0.53, 0.40) is below persistence's (0.86, 0.54), and 89% of its rollouts stay below VRMSE 1 through step 30. AViT, despite a similar one-step error, degrades fastest (0.80, 0.65; 63%): averaged over the rollouts from frames 80 and 140, its error after 30 steps (0.72) is no better than persistence's (0.70), and from frame 20, during onset, it grows to 5.4. TFNO lies in between. Persistence keeps more rollouts below the thresholds because its error levels off near 0.7, while it is 2–5 times less accurate than every model over the first steps from developed flow (Fig. 3a). Under the paper's frame-0 protocol all three models land in the "> 10" range; CNextU-Net (15 / 30) is below the released CNextU-Net (36 / 35).

With only one hour of training (Tables 3 and 4; the four-hour runs' checkpoints after one hour). The ranking differs from four hours, and no model is best on every count. AViT has the lowest one-step error over all windows (0.39, against 0.47 for TFNO and 0.73 for CNextU-Net) and during onset; in developed convection CNextU-Net (0.081) and AViT (0.085) are close and TFNO trails (0.120). Over rollouts TFNO is the most stable: 60% of its rollouts stay below VRMSE 1 through step 30, against 34% for the other two, and from developed flow it stays more accurate than persistence for the first 21 steps, against 14 for CNextU-Net and 12 for AViT. CNextU-Net, the best model after four hours, is the least reliable after one: its two seeds differ most (all-window VRMSE 0.85 and 0.60), and none of one seed's rollouts stays below VRMSE 1, against 69% for the other. With one hour of training, AViT is the better choice for one-step predictions and TFNO for multi-step forecasts; after 30 steps none of the three is more accurate than persistence.

Accuracy against cost (Fig. 4, Table 5). Because data and compute are the limiting resources, we set each model's accuracy against what it costs to train and to use.

Model	VRMSE after 30 steps from frame 20 / 80 / 140	Fraction staying below threshold through step 30: 1 / 2 / 5	Paper protocol, steps 6–12 / 13–30 (normalised)
CNextU-Net, 4 h	1.78 / **0.53** / **0.40**	**0.89** / **0.94** / **0.98**	**15** / **30**
TFNO, 4 h	1.79 / 0.59 / 0.45	0.75 / 0.85 / 0.91	51 / 77
AViT, 4 h	5.42 / 0.80 / 0.65	0.63 / 0.80 / 0.87	75 / 167
CNextU-Net, 1 h	3.60 / 1.09 / 1.04	0.34 / 0.74 / 0.79	202 / 148
TFNO, 1 h	**2.67** / **0.90** / **0.78**	**0.60** / **0.77** / **0.86**	**81** / **116**
AViT, 1 h	4.67 / 1.11 / 0.98	0.34 / 0.74 / 0.83	129 / 184
Released CNextU-Net (100%)			36 / 35
Persistence	0.68 / 0.86 / 0.54	0.94 / 1.00 / 1.00	0.27 / 0.50

Table 4: Rollouts on the test set, physical-field VRMSE unless stated, mean over the two seeds. Middle column: fraction of the 525 rollouts (175 trajectories × 3 start frames) whose error stays below each threshold at every step through step 30. Right: the paper's Table 3 protocol (published: > 10 in both windows); bold: best of our models at each budget.

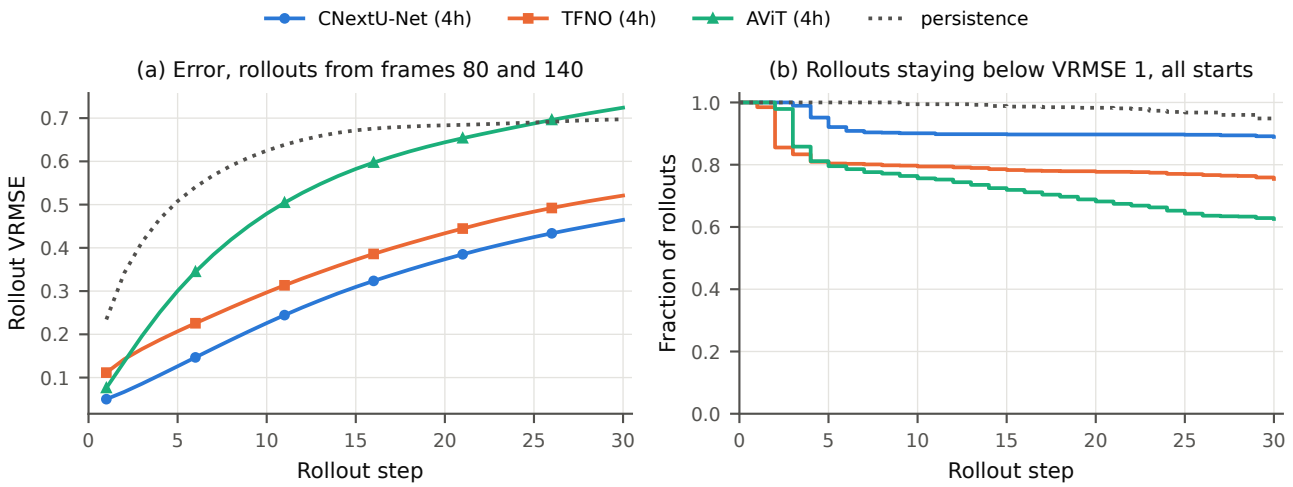


Figure 3: Rollouts of our 10% models at 4 hours (seed mean) and persistence. (a) Physical-field VRMSE at each step, mean over the rollouts from frames 80 and 140 (developed convection). (b) Fraction of all 525 rollouts that have stayed below VRMSE 1 up to each step (Kaplan–Meier survival).

- *Training time* (Fig. 4a). From one to four hours the developed-convection error falls by 38% for CNextU-Net (0.081 to 0.050), 12% for AViT (0.085 to 0.075) and 9% for TFNO (0.120 to 0.109). At one hour CNextU-Net and AViT are close; CNextU-Net pulls ahead only with more training, while TFNO has nearly plateaued.

- *Cost to train* (Fig. 4b). Every run trained for the same 4 hours, but the GPU time it held differed. AViT is the most efficient on the GPU (six times as many optimizer steps as CNextU-Net, a seventh of the memory), but it consumes 330 samples per second, more than our shared storage delivers (∼98; Fig. 7). Each AViT run therefore held a GPU for 11–12 hours, against 6.9–7.1 for TFNO and 5.2–6.0 for CNextU-Net. On faster storage this ranking would change; the demand per GPU-second is the portable number. CNextU-Net's 61 GB of training memory at batch 64 needs an 80 GB GPU or a smaller batch.

- *Cost to use* (Fig. 4c). For a scientist running forecasts, inference is the recurring cost. Every model predicts the next 512 × 128 frame in 4–8 ms and a 30-step forecast (7.5 time units) in 135–249 ms, using under 0.5 GB of GPU memory. For one forecast at a time TFNO is the fastest; AViT takes 1.2 times and CNextU-Net 1.8 times as long. CNextU-Net's extra 114 ms per forecast buys a 30-step

error of 0.47 instead of TFNO's 0.52, while AViT is slower than TFNO and, after 30 steps, no more accurate than persistence. The ranking changes for many forecasts at once: in batches of 64 the forward pass costs 0.89 ms per sample for AViT, 3.15 ms for TFNO and 6.74 ms for CNextU-Net, so for ensembles or many initial conditions AViT is 3.5 times cheaper per forecast than TFNO and 7.6 times cheaper than CNextU-Net. For context, The Well reports that generating the data with the Dedalus solver took about 6,000 to 50,000 s per parameter setting on CPU nodes [13]. That figure covers many trajectories with an unstated parallel layout, so it is not comparable to a per-forecast time; the solver baseline in the plan would measure that comparison directly.

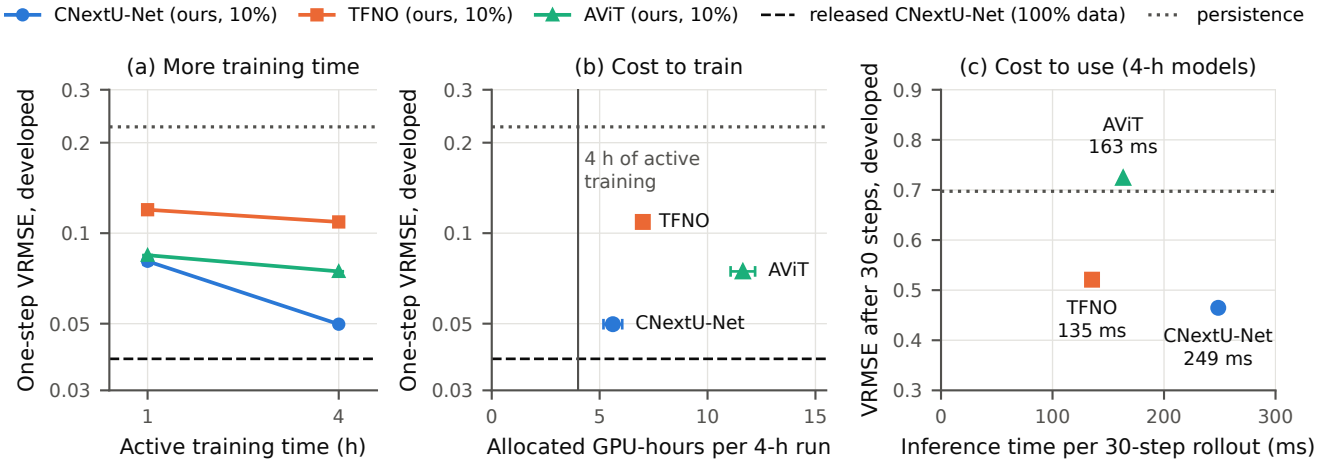


Figure 4: Accuracy against cost, 10% data (seed mean; bars: range over seeds). (a) One-step test VRMSE in developed convection at the 1-hour and 4-hour checkpoints. (b) The same error after 4 hours against the GPU time each run held, including data waits and overhead; vertical line: the 4 hours of active training. (c) Rollout VRMSE after 30 steps from frames 80 and 140 against the time to compute one 30-step forecast (batch 1, A100). Dashed: the released CNextU-Net (100% data); dotted: persistence.

	Training (4-hour budget, batch 64)				Inference			
Model	Optimizer steps	Epochs	Peak memory, alloc. (reserved)	GPU time held per run	per step, batch 1	30 steps, batch 1	per sample, batch 64	Memory, batch 1
CNextU-Net	12,122–12,194	28	60.9 (64.8) GB	5.2–6.0 h	8.3 ms	249 ms	6.74 ms	0.31 GB
TFNO	26,339–26,352	61	48.6 (56.9) GB	6.9–7.1 h	4.4 ms	135 ms	3.15 ms	0.44 GB
AViT	74,978–75,060	175	8.8 (8.9) GB	11.1–12.2 h	5.4 ms	163 ms	0.89 ms	0.23 GB

Table 5: Cost to train and to use, on one A100 80 GB in fp32. Peak training memory: allocated by tensors (reserved by PyTorch's allocator). GPU time held: allocated time of the training job, including data waits, start-up, validation and checkpoints (Appendix E). Per sample, batch 64: forward pass in the training profile (with gradient tracking, an upper bound for inference). Inference memory: peak allocated at batch 1.

Physical fidelity (4-hour checkpoints; Fig. 5). The kinetic-energy spectrum shows at which scales each model's errors lie. It is a complementary diagnostic, not the paper's binned spectral error, and it describes how energy is distributed over scales, not where structures are: a model can match the spectrum with plumes in the wrong places. The values below are medians over the 35 regimes and both seeds of the predicted-to-true ratio, taken per wavenumber at steps 1, 10 and 30 (the steps evaluated). CNextU-Net stays within 0.80–1.57 below $k_x = 100$ and reaches 190–530 at the highest wavenumbers. TFNO is too smooth: between $k_x \approx 25$ and 100 its ratio falls to 0.33 after one step and 0.09 after 30, and it reaches 9–50 at the highest wavenumbers. AViT stays within 0.97–1.34 below $k_x = 100$ after one step, but after 30 steps it ranges from 0.55 to 21 there and reaches about 5×10^4 at the highest wavenumbers. The TFNO deficit is consistent with its spectral layers keeping only the 16 lowest Fourier modes in each direction

($k_x \lesssim 25$) [5], and AViT's growth with the patch-shaped artifacts in Fig. 6; neither is established as the cause. The excess small-scale energy of CNextU-Net and AViT is typical of surrogates trained with MSE, which weights each scale by its energy and so barely constrains low-energy small scales (the spectral bias of neural networks [7]); such surrogates are known to neglect non-dominant, often high-frequency, components [8]. Persistence repeats its last input field, so its predicted spectrum is fixed within a rollout; changes in its ratio reflect the evolving true spectrum. Ratios at the smallest scales should also be read alongside absolute spectra, because very small true energy can magnify the ratio.

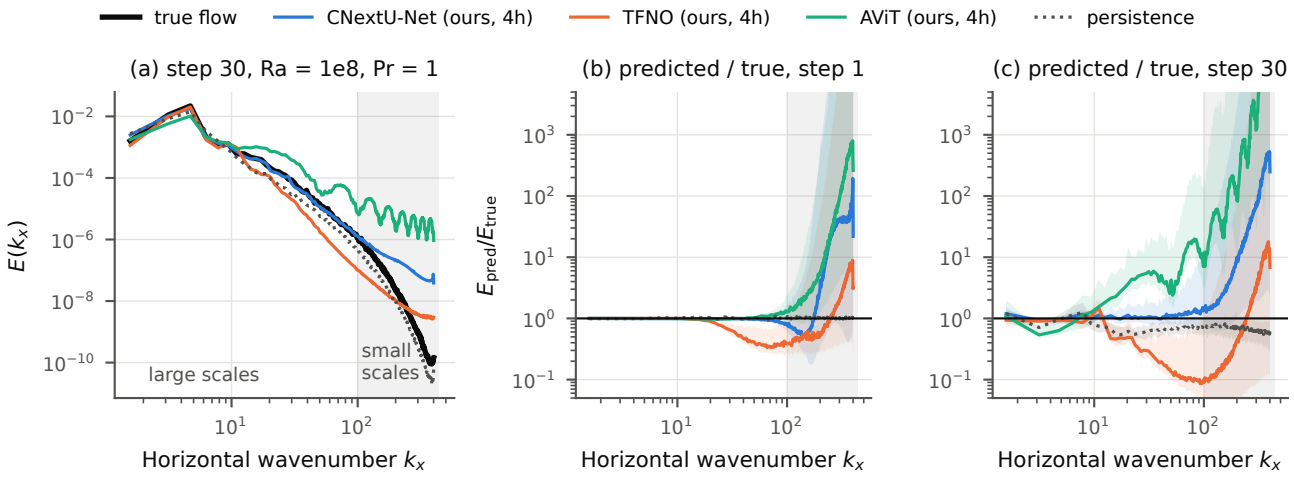


Figure 5: Horizontal kinetic-energy spectra. (a) True and predicted spectrum after 30 rollout steps for one regime ($\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$). (b, c) Predicted over true spectrum after 1 and 30 steps, per wavenumber: median over the 35 regimes (band: middle 50%); 1 is exact. Shaded: small scales ($k_x > 100$).

What the predictions look like (Fig. 6; $\mathrm{Ra} = 10^{10}$ in Appendix F). The figure shows buoyancy, while the spectra are computed from velocity, so the two are related qualitative evidence rather than measurements of the same field. At $\mathrm{Ra} = 10^8$ CNextU-Net follows the true plumes through 30 steps. TFNO blurs them and builds a spurious warm layer along the bottom wall. By step 12 AViT shows rectangular artifacts at the size of its 16×16-pixel patches. At $\mathrm{Ra} = 10^{10}$ the true flow is far finer than any of the three reproduces; CNextU-Net keeps the large-scale plume pattern but smooths the small structures.

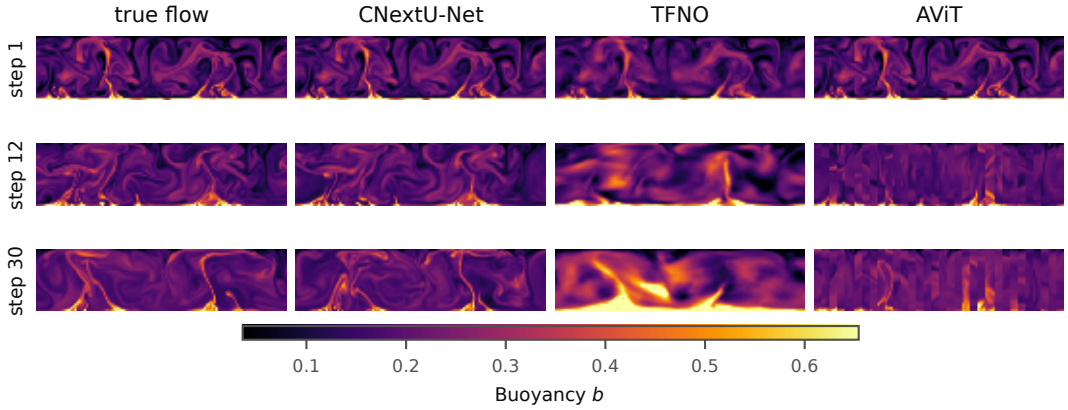


Figure 6: Buoyancy on a test trajectory at $\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$, rolled out from frame 80: the true flow and our 10% / 4-hour models (seed 0) after 1, 12 and 30 steps. Bright: warm; dark: cold. One colour range is used for all panels: the 1st to 99th percentile of the true field away from the walls, so the hot bottom wall saturates.

Choosing a model under limited data and compute (Table 6). No family is best on every count.

CNextU-Net is the most useful forecaster in developed convection at 10% data and 4 hours, at the price of the slowest inference and the most training memory; TFNO is the fastest for single forecasts but too smooth; AViT is the cheapest on the GPU, in training and for batched forecasts, but needs fast storage and does not stay accurate over a rollout.

If the priority is	Best of the three	Trade-off
Accurate forecasts of developed convection that stay stable over 30 steps	CNextU-Net	Slowest inference (249 ms per 30 steps); 61 GB training memory at batch 64; gains most from longer training
Fastest single forecast (batch 1)	TFNO (135 ms per 30 steps)	Too smooth: loses energy between $k_x \approx 25$ and 100; largest one-step error in developed convection; gains least from longer training
Least training memory, most optimizer steps per GPU-hour, cheapest batched forecasts	AViT (8.8 GB; 0.89 ms per sample at batch 64)	Needs storage delivering ∼330 samples/s; rollouts degrade fastest, no better than persistence after 30 steps
Convection onset (first 50 frames)	None: persistence	Every model is less accurate than copying the last frame
Only one hour of training	AViT (one step); TFNO (rollouts)	AViT: lowest one-step error (0.39); TFNO: 60% of rollouts stay below VRMSE 1, against 34%; CNextU-Net varies most between seeds; none beats persistence after 30 steps

Table 6: Which model to use under which constraint, from the 10% results, after four hours of training unless the row says otherwise. These hold for these configurations and budgets and may change with all the training data.

5 Findings Under Limited Data and Compute

The main findings at 10% data hold for these configurations and budgets (Table 6):

- **Accuracy.** After 4 hours CNextU-Net is the most accurate in developed convection (one-step VRMSE 0.050) and the most stable over 30-step rollouts. With a tenth of the data and 4 hours on an A100, its error is 1.3 times that of The Well's released CNextU-Net, trained on all the data for 12 hours on an H100 (0.038); the two also differ in schedule and hardware, so this is not an estimate of what more data buys. TFNO's predictions are too smooth; AViT matches CNextU-Net over all windows but degrades fastest over a rollout.
- **With only one hour of training** the ranking differs. AViT has the lowest one-step error and TFNO the most stable rollouts, while CNextU-Net, the best after four hours, varies most between seeds. CNextU-Net gains most from the extra three hours: its error in developed convection falls by 38%, AViT's by 12% and TFNO's by 9%.
- **Cost to train.** In the same 4 hours AViT takes six times as many optimizer steps as CNextU-Net with a seventh of its memory, but it reads data faster than our shared storage delivers, so a run held a GPU for 11–12 hours, against 5–6 hours for CNextU-Net.
- **Cost to use.** Each model forecasts 30 steps (7.5 simulation time units) in at most a quarter of a second on one A100, using under 0.5 GB of GPU memory. For one forecast at a time TFNO is the fastest (135 ms), then AViT (163 ms) and CNextU-Net (249 ms), so the most accurate model is 1.8 times slower than the fastest. For batched forecasts AViT is the cheapest per sample (0.89 ms per step, against 3.15 for TFNO and 6.74 for CNextU-Net).
- **Negative results.** No model beats copying the last frame (persistence) during convection onset, and after 30 steps from developed flow AViT is no better than persistence.

- **Caveats.** Two seeds on one fixed 10% subset do not measure how much the choice of trajectories matters. Training in fp32 for every model favours TFNO, since bf16 would speed up CNextU-Net (1.7 times) and AViT (2.2 times) but is not available for our TFNO. The learning rate is the best of a short search, at the top of the tested grid.

6 Revised Plan

The *core comparison* remains three families, 10% and 100% training data, two seeds, and checkpoints at one and four hours. The 100% runs complete the data axis, so the accuracy–cost comparison of Fig. 4 and Table 6 can be repeated with all the data and the value of ten times more data measured at each budget. The main changes are:

- **Budget definition.** Experimental budgets count active GPU training time. Allocated GPU time, data waits and SU are reported separately so that storage costs remain visible.
- **Evaluation.** Report the paper protocol for reference comparisons alongside the physical-field VRMSE, separate onset from developed convection, and report the failure horizon at thresholds 1, 2 and 5.
- **Core resource estimate.** The 48 hours of active training are unchanged; the working estimate is approximately 135 allocated GPU-hours for the core study and its supporting work (Table 12, Appendix E), instead of the abstract's approximately 100. With all 10% work complete, Slurm accounting gives 72.6 allocated GPU-hours: 54.2 on the paid partition, billed 2,160 SU (40 SU per allocated hour for jobs with 8 cores, 36 for the 4-core pipeline check), and 18.3 on the free partition, which is not billed.
- **Optional, outside the estimate.** 24-hour runs of CNextU-Net and TFNO with the paper's learning rates (roughly 60–90 allocated GPU-hours; not an exact replication of the paper's 12-hour H100 runs), and a Dedalus [11] solver baseline on CPUs, which first needs a feasibility check; it would set the models' accuracy and forecast time against the solver's for the same 30-step forecasts. A second draw of the 10% subset (one run per model, about 25 allocated GPU-hours) would measure how much the choice of trajectories matters, and batched inference timing can be added to the 100% evaluations at little cost.

Remaining deliverables. The 10% evaluation is complete for all three models and both seeds, including failure horizons, spectra, training cost, inference time and memory. Remaining for the final report: the 100% runs and the data–compute comparison (10% against 100% at 1 and 4 hours), with accuracy–cost curves at both data fractions; field-wise VRMSE and the Nusselt-number error per model; and the flow snapshots for the 100% runs.

Next steps, in priority order. (1) Improve data-loading robustness and assess node-local staging; (2) complete the six 100% runs (estimated approximately 2,400 SU) and the core data–compute comparison; (3) complete the promised metrics and the reproducible final report. Only after core completion, consider the abstract's extensions in their original order: input-noise robustness, a third seed, a 30% fraction, and ConvLSTM. The optional runs above should not delay the core deliverables.

Appendix A: Detailed Training Recipe

Sources of the reference settings: the paper (learning rates, 12-hour H100 budget) and The Well's training configuration at commit `e63dec8`:

- `configs/optimizer/adam.yaml`: AdamW, weight decay 10^{-4} (the paper's text says 0.01; we follow the code);
- `configs/data/rayleigh_benard.yaml`: batch 64;
- `configs/trainer/defaults.yaml`: 200 epochs;
- `configs/lr_scheduler/cosine_with_warmup.yaml` with `train.py`: 5 warmup epochs from 0.1·lr, then cosine to 0.1·lr.

Our software: The Well `e63dec8`, neuraloperator 0.3.0, PyTorch 2.8.0 (CUDA 12.8), Python 3.13; our repository at `f42b2e2` (training) and `4c0d3a8` (AViT resume, evaluation).

	This study	The Well paper / code
Activation	GELU (all three models)	GELU (same model code)
Optimizer	AdamW, $\beta = (0.9, 0.999)$, weight decay 10^{-4}, no clipping	AdamW, weight decay 10^{-4} in the code (0.01 in the text), no clipping
Loss	one-step MSE, normalised units, equal field weights	same
Batch size, precision	64, fp32	64, fp32
Learning rate	10^{-3} for all, from our search over $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}\}$	per model from $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}, 5\cdot10^{-3}, 10^{-2}\}$: $5\cdot10^{-4}$ (CNextU-Net), 10^{-4} (TFNO)
Schedule	500 warmup steps, cosine to 0 over the 4-hour budget	5 warmup epochs, cosine to 0.1·lr over 200 epochs, stopped at 12 h
Budget, hardware	1 h and 4 h of active GPU training on one A100	12 h on one H100, best validation epoch
Training data	10% of the trajectories (100% next)	100%

Table 7: Training recipe: ours against the paper's.

Released CNextU-Net's schedule. By our estimate (our A100 throughput, with an H100 at most three times faster), the paper's 12-hour runs completed at most about 25 of their 200 scheduled epochs, so their learning rate was still above 95% of its peak when training stopped, whereas ours decays to zero within the budget.

Validation VRMSE after 20 min	10^{-4}	$5\cdot10^{-4}$	10^{-3}
CNextU-Net	0.841	0.469	**0.408**
TFNO	0.603	0.358	**0.302**
AViT	0.646	0.332	**0.257**

Table 8: Learning-rate search.

Appendix B: Metric Details

Physical-field VRMSE and the paper protocol. Without ε the two would be identical, since the ratio is invariant to an affine rescaling of each field. With ε, normalising multiplies s_f^2 by $1/\sigma_f^2$ (16–45), so the floor shrinks from 10^{-7} to the equivalent of $\varepsilon\sigma_f^2 \approx 2\text{–}6\times10^{-9}$. During the first ∼50 frames the velocity

is nearly zero (exactly zero at $t = 0$) and its spatial variance is comparable to 10^{-7}, so the physical-field score is damped there and the normalised one is not. This is the whole difference between the released CNextU-Net's 0.669 (paper protocol) and 0.224 (physical fields): on early windows they score 2.510 and 0.766, while from frame 50 on both give 0.038.

Kinetic-energy spectrum. At each grid row, the two velocity components are Fourier-transformed along the periodic x axis; $E(k_x)$ is the one-sided energy $\frac{1}{2}(|\hat{u}|^2 + |\hat{w}|^2)$ at $k_x = 2\pi n / L_x$ ($L_x = 4$), averaged over the Chebyshev-spaced rows with quadrature weights, so that summing over k_x gives the domain-mean kinetic energy. Spectra are averaged over the rollouts of each regime (5 trajectories $\times$ 3 start frames) at steps 1, 10 and 30, separately for predictions and truth.

Inference time. Measured as a scientist would use a model: one forecast at a time (batch 1), fp32, on one A100 80 GB, with the input already on the GPU. After 10 warm-up predictions we time 50 one-step predictions (reported per step) and one full 30-step autoregressive rollout, with a CUDA synchronisation before each clock stops. At batch 1 the GPU is far from fully used, so this latency includes per-call overhead as well as computation. For many forecasts at once, such as ensembles, we also report the forward-pass time per sample in a batch of 64, measured during profiling (fp32, with gradient tracking as in training, so an upper bound for inference).

Appendix C: TFNO Forward Pass and the Released TFNO

In The Well's code, commit `2d8d7eaf` (2024-06-03, "Added gradient checkpointing") called TFNO's Fourier blocks without keeping their output, so the model computed only its lifting and projection layers; commit `e4301602` (2025-11-17, "actually use the TFNO blocks") restored the assignment (`x =` before the block call) [12]. The paper's TFNO results and the released TFNO checkpoint (uploaded 2025-03-28) date from that period. For the pipeline check we scored the released TFNO weights with the forward pass they were trained with (Fourier blocks skipped): it reproduces the published value, while with the corrected forward pass the same weights score ≈10.5. Our own runs use The Well at commit `e63dec8`, which includes the fix. Upstream issue #49 reports related inconsistencies between released checkpoints and published numbers [14].

Model	Paper-protocol one-step VRMSE (published)	Physical-field one-step all / onset / developed	Rollout from frame 0, steps 6–12 / 13–30
Released TFNO (paper-era forward)	0.663 (0.657)	0.300 / 0.623 / 0.190	142 / 169 (published > 10)
Our TFNO (10% data, 4 h)	0.488	0.209 / 0.502 / 0.109	51 / 77
Persistence	0.202	0.199 / 0.121 / 0.226	0.27 / 0.50

Table 9: The released TFNO, scored by our pipeline on the test split, against our TFNO (seed mean) and persistence.

Why the released TFNO tracks persistence (Fig. 1). With its Fourier blocks skipped, the model is only its lifting and projection layers, which act on each grid point separately: every output pixel is computed from the same pixel's four input frames (and at most its position), with no information from neighbouring pixels. Such a model cannot move structures, such as rising plumes or flow carried by the convection rolls, so what it learns is close to copying each pixel's last value with a small correction from its recent trend: persistence with a little extrapolation. In developed convection it is therefore only slightly better than persistence (physical-field VRMSE 0.190 against 0.226). During onset the fields barely change, so copying is nearly exact, while the model's small per-pixel adjustments are divided by a near-zero spatial variance and become large relative errors. The published all-window value (0.657) hides this, because every model scores poorly on the onset windows. Our TFNO, which uses its Fourier layers, is better than the released TFNO everywhere (developed 0.109 against 0.190; paper protocol 0.488

against 0.663) despite 10% of the data and 4 hours.

Appendix D: Further Pipeline-Check Observations

- **Rollout error varies substantially with Rayleigh number.** At Ra = 10^{10} the released CNextU-Net's median over steps 13–30 is 1.7; at 10^6 it is 69.
- **The fixed test evaluation is the clearer reproduction target.** The paper's validation values use 20 random batches per epoch; exact agreement would additionally require the same sampled batches and evaluation state.

Appendix E: Resource Accounting

Timer. Active training time is measured per optimizer step with a host-side monotonic clock (Python's `perf_counter`), from the moment the data loader returns the batch to just after a CUDA synchronisation (`torch.cuda.synchronize`) following the optimizer update. It includes the host-to-device copy of the batch (pinned memory, non-blocking), setting the learning rate, the forward pass, the loss, the backward pass, the optimizer step and the host-side launch overhead of these operations. It excludes the wait for the next batch (logged as data-wait time), reading the loss value, logging, validation, checkpoint writes and start-up. After a crash the clock resumes from the last saved checkpoint, so lost steps are not counted. A milestone checkpoint is written after the first step at which the cumulative time reaches 1 h or 4 h (overshoot below one step).

Validation sample. The 64 validation windows are fixed and evenly spaced over the 34,300 one-step windows of the validation split (175 trajectories × 196), taken in the dataset's order (regime, trajectory, time): indices round(linspace$(0, 34299, 64)$). They cover all 35 regimes (one or two windows each) and 64 different trajectories; 18 of them (28%) are onset windows, close to the 26% of all windows. The learning-rate search used 512 windows chosen the same way (all 35 regimes, all 175 trajectories, 25% onset); the two samples share 8 windows. This monitoring sample does not replace the full test evaluation.

Billing. Paid jobs are billed 32 SU per GPU-hour plus 1 SU per core-hour of allocated time (40 SU per hour for one A100 and 8 cores); the free partition is not billed. Runs that started on the free partition and resumed on the paid one are billed only for the paid part.

Pilot against actual. The pilots predicted 4 h of GPU training to take 6.0 / 8.0 / 9.7 h of wall time (CNextU-Net / TFNO / AViT) at the stall rate observed then; the 10% runs took 5.2–6.0 / 6.9–7.1 / 11.1–12.2 h.

	Validation VRMSE (seed 0 / 1)		Epochs	Allocated time	SU
Model	1 h	4 h	in 4 h	per run	per run
CNextU-Net	1.043 / 0.772	0.167 / 0.178	28	5.2–6.0 h	189–191
TFNO	0.508 / 0.524	0.310 / 0.325	61	6.9–7.1 h	274–286
AViT	0.468 / 0.418	0.161 / 0.167	175	11.1–12.2 h	443–488

Table 10: The 10% runs (140 training trajectories). Validation: 64 fixed windows, physical-field VRMSE. SU covers the paid partition only: CNextU-Net's first attempts ran on the free partition, so its SU corresponds to 4.8 h of its 5.2–6.0 h.

Hours per run	CNextU-Net	TFNO	AViT
10% runs, measured			
Queue wait before training started	0.1–0.9	0.5–0.7	6.7–8.8
GPU training (the budget)	4.0	4.0	4.0
Waiting for data	0.8–1.1	2.5–2.8	6.1–7.3
Start-up, and steps replayed after a preemption or crash[a]	0.13–0.65	0.14–0.15	0.68–0.69
Validation (64 windows every 30 minutes)	0.01–0.02	0.02–0.10	0.18
Checkpoint writes	0.23–0.24	0.08–0.15	0.03–0.13
Allocated time of training	**5.2–6.0**	**6.9–7.1**	**11.1–12.2**
Test evaluation job (1-hour and 4-hour checkpoints)	0.87–0.95	0.57	0.34–0.38
Queue wait before the evaluation started	1.0–14.0	1.9–2.4	3.0–3.3
Training attempts (seed 0 / seed 1)	2 / 2	1 / 1	2 / 2
100% runs, projected			
Waiting for data	1.0–2.0	3.0–4.5	9.5–12.5
Allocated time of training	**5.5–6.5**	**7.5–9**	**14–17**
Test evaluation job	∼0.9	∼0.6	∼0.4

Table 11: Time per run (range over the two seeds). [a]A run that starts once spends about 0.15 h in start-up (environment, data index, model build); the larger values belong to runs with two attempts, which also replayed the steps since their last saved checkpoint. Projection for 100%: the shared file system delivers at most ∼98 samples/s, below TFNO's (116) and AViT's (330) demand, and the full training split (∼290 GB) no longer fits in a node's page cache; start-up, validation, checkpoint writes and evaluation as measured. Queue waits depend on the cluster's load and are not projected.

Component	Allocated GPU-hours
Six 10% runs, completed (24.0 of them active training)	49.8
Learning-rate search, profiling, pipeline check, environment checks, first replication try	18.5
Test evaluation of the six 10% runs and persistence (completed)	4.2
Six 100% runs (projected, 24.0 active; Appendix E)	∼54–65
Test evaluation of the 100% runs	∼4.5
Core study	∼131–142

Table 12: Allocated GPU-hours of the core study: completed parts from Slurm accounting (2026-11-11), the rest projected. No contingency is included; the abstract's ∼100 hours included ∼30, and a similar margin would bring the total to ∼160–170.

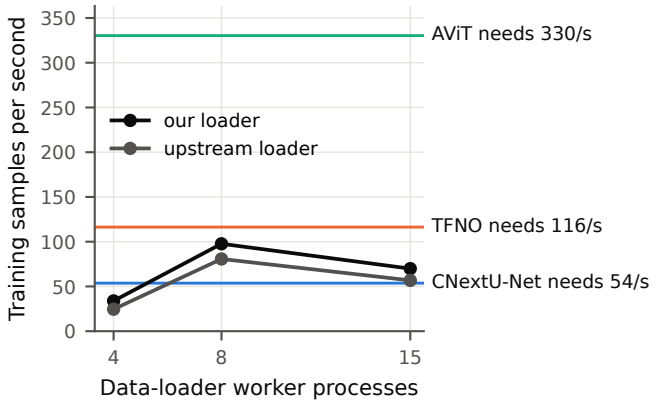


Figure 7: Training samples per second read from the shared file system by one node, against the rate at which each model consumes them on the GPU (horizontal lines).

Appendix F: Additional Figures

Test error by window (Fig. 8) and flow snapshots at $\mathrm{Ra} = 10^{10}$ (Fig. 9).

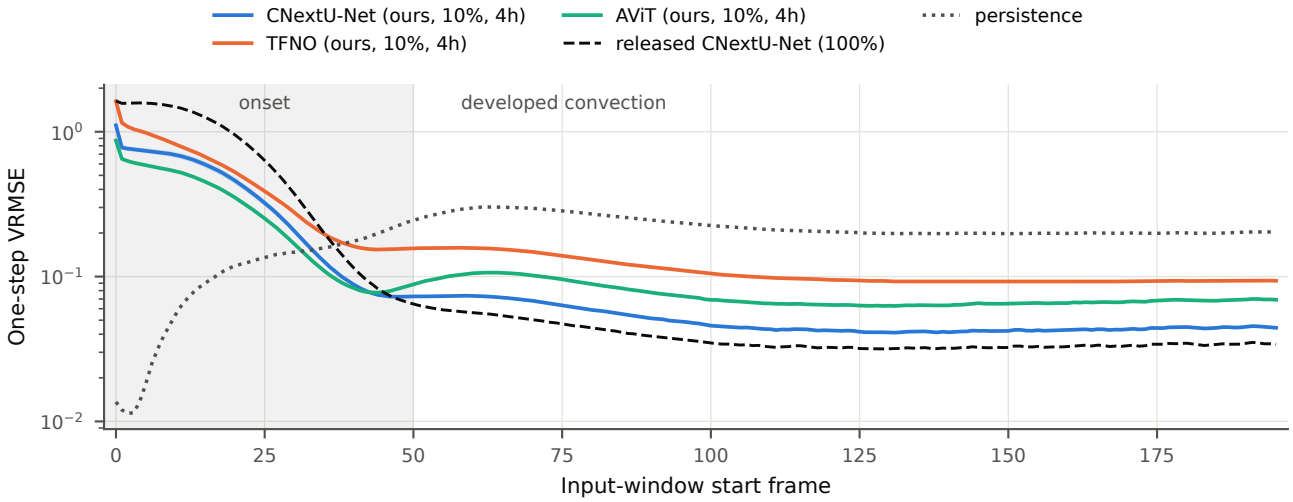


Figure 8: One-step test VRMSE (physical fields) by window, mean over the 175 test trajectories: our 10% models at 4 hours (seed mean; band: range over seeds), the released CNextU-Net trained on all the data, and persistence. Shaded: onset windows (first input frame < 50; see Fig. 1).

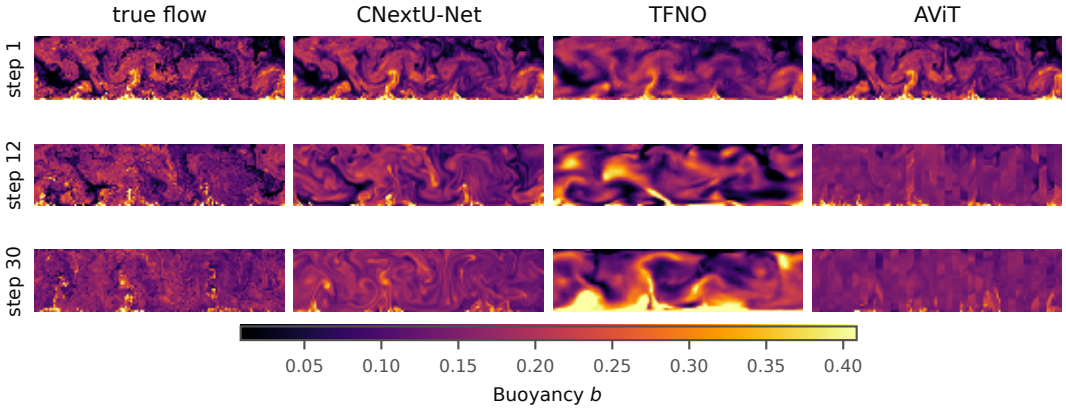


Figure 9: As Fig. 6, at $\mathrm{Ra} = 10^{10}$, $\mathrm{Pr} = 1$.

References

[1] Ohana, R., et al. "The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning." NeurIPS, Datasets and Benchmarks Track, 2024.
https://arxiv.org/abs/2412.00568

[2] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. "A ConvNet for the 2020s." CVPR, 2022.
https://arxiv.org/abs/2201.03545

[3] Kossaifi, J., Kovachki, N., Azizzadenesheli, K., and Anandkumar, A. "Multi-Grid Tensorized Fourier Neural Operator for High-Resolution PDEs." TMLR, 2024.
https://arxiv.org/abs/2310.00120

[4] McCabe, M., et al. "Multiple Physics Pretraining for Spatiotemporal Surrogate Models." NeurIPS, 2024.
https://arxiv.org/abs/2310.02994

[5] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. "Fourier Neural Operator for Parametric Partial Differential Equations." ICLR, 2021.
https://arxiv.org/abs/2010.08895

[6] Gupta, J. K., and Brandstetter, J. "Towards Multi-spatiotemporal-scale Generalized PDE Modeling." arXiv:2209.15616, 2022.
https://arxiv.org/abs/2209.15616

[7] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F. A., Bengio, Y., and Courville, A. "On the Spectral Bias of Neural Networks." ICML, 2019.
https://arxiv.org/abs/1806.08734

[8] Lippe, P., Veeling, B. S., Perdikaris, P., Turner, R. E., and Brandstetter, J. "PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers." NeurIPS, 2023.
https://arxiv.org/abs/2308.05732

[9] Brandstetter, J., Worrall, D., and Welling, M. "Message Passing Neural PDE Solvers." ICLR, 2022.
https://arxiv.org/abs/2202.03376

[10] McCabe, M., Harrington, P., Subramanian, S., and Brown, J. "Towards Stability of Autoregressive Neural Operators." TMLR, 2023.
https://arxiv.org/abs/2306.10619

[11] Burns, K. J., Vasil, G. M., Oishi, J. S., Lecoanet, D., and Brown, B. P. "Dedalus: A flexible framework for numerical simulations with spectral methods." Physical Review Research 2, 023068, 2020.
https://arxiv.org/abs/1905.10388

[12] The Well source code, commits `2d8d7eaf9eb46b96f215abf39c94b6be4bf9ea5c` (2024-06-03) and `e4301602e57f2dddeaa87856dda49bbba47c663b` (2025-11-17).
https://github.com/PolymathicAI/the_well/commit/2d8d7eaf9eb46b96f215abf39c94b6be4bf9ea5c

https://github.com/PolymathicAI/the_well/commit/e4301602e57f2dddeaa87856dda49bbba47c663b

[13] The Well, `rayleigh_benard` dataset card, commit `e63dec8`.
https://github.com/PolymathicAI/the_well/blob/e63dec8/datasets/rayleigh_benard/README.md

[14] The Well, issue #49, "Inconsistent Pre-trained Baseline Performance" (open since 2025-04-21).
https://github.com/PolymathicAI/the_well/issues/49